

# nodeIO — README del programador

---

Firmware del **Nodo IO** (responder LoRa) para **Heltec WiFi LoRa 32 V3** (ESP32-S3 + SX1262). Atiende peticiones de un maestro por LoRa: reporta 4 entradas analógicas + 4 digitales y escribe 4 relés. Se configura por portal cautivo (NVS).

Proyecto hermano: `../nodeIO_master` (maestro de prueba).

---

## 1. Toolchain

- PlatformIO (`platform = espressif32`, `framework = arduino`, board `heltec_wifi_lora_32_v3`).
- Dependencias (`platformio.ini`):
  - `ropg/Heltec_ESP32_LoRa_v3` — wrapper "heltec\_unofficial": trae **RadioLib** (radio `SX1262`), **SSD1306Wire** (OLED `display`), **HotButton** (`button`) y helpers de energía.
  - `bakercp/CRC32` — CRC32 del payload.
- `WiFi`, `WebServer`, `DNSServer`, `Preferences` vienen en el core `arduino-esp32`.

```
pio run          # compilar
pio run -t upload # flashear
pio device monitor -b 115200
```

---

## 2. Arquitectura

| Módulo                               | Responsabilidad                                                                                                                                                                                                              |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>src/main.cpp</code>            | Máquina de estados <code>MODE_NORMAL</code> / <code>MODE_PORTAL</code> , splash OLED, render de estado, long-press de <code>BUTTON_1</code> , botón PRG local. <b>Único TU que incluye <code>heltec_unofficial.h</code>.</b> |
| <code>src/io.{h,cpp}</code>          | Mapa de pines, init, ISR compartida de entradas digitales/botones, lectura de AI/DI, control de relés con máscara de habilitación + estado seguro + pulsos.                                                                  |
| <code>src/node_config.{h,cpp}</code> | <code>struct NodeConfig</code> persistida en NVS (namespace <code>nodeio</code> , un blob + <code>magic</code> ). Sustituye los stubs <code>rescueFlashConfig()/saveFlashConfig()</code> del scaffold original.              |
| <code>src/portal.{h,cpp}</code>      | Portal cautivo: <code>WiFi.softAP</code> + <code>DNSServer</code> (:53, *) + <code>WebServer</code> (:80) con formulario. <code>POST /save</code> valida, persiste y <code>ESP.restart()</code> .                            |

| Módulo                              | Responsabilidad                                                                                                                                                                                                                                                                                           |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>src/lora_proto.{h,cpp}</code> | Protocolo responder: <code>begin()</code> desde <code>cfg</code> , RX por interrupción ( <code>setDio1Action</code> ), verificación CRC32, dispatch de <code>RD</code> / <code>WR</code> / <code>WP</code> / <code>PING</code> , construcción de <code>ST</code> / <code>ERR</code> / <code>PONG</code> . |
| <code>src/images.h</code>           | XBM del splash ( <code>poweredBy</code> , <code>aysafi_Logo_bits</code> ).                                                                                                                                                                                                                                |
| <code>PROTOCOL.md</code>            | Especificación de la trama y los comandos.                                                                                                                                                                                                                                                                |

## Regla de include de `heltec_unofficial.h`

El header **define objetos globales y funciones no-inline** (`radio`, `display`, `button`, `both`, `heltec_*`). Si se incluye en más de un `.cpp` → error de símbolos duplicados en el enlace. Por eso:

- Solo `main.cpp` lo incluye.
- `lora_proto.cpp` hace `#include <RadioLib.h>` y `extern SX1262 radio;` para usar la instancia global sin redefinirla.
- Todo el dibujo en OLED vive en `main.cpp` (los demás módulos exponen estado, no dibujan).

## 3. Flujo de ejecución

```

setup(): heltec_setup() -> configLoad() -> ioInit(cfg.relayEnable,
        cfg.relaySafe)
        -> splash()
        -> configStored() ? loraBegin() : enterPortal()

loop(): heltec_loop() // HotButton
        MODE_NORMAL: BUTTON_1 > 3 s -> enterPortal()
        loraLoop() // atiende 1 paquete RX y responde
        ioServicePulses(cfg.relaySafe) // cierra pulsos vencidos
        serviceLocalInputs() // PRG conmuta relé 1, limpia flags
DI
        drawStatusScreen() // OLED throttled 250 ms
        MODE_PORTAL: portalLoop() + drawPortalScreen()

```

RX es **half-duplex**: la ISR marca `rxFlag`; `loraLoop()` hace `getPacketLength()` + `readData()`, re-arma con `startReceive()`, valida CRC y despacha. La respuesta se transmite con `radio.transmit()` y se vuelve a `startReceive()`.

## 4. Protocolo (resumen)

```
"<dst>,<src>,<seq>,<cmd>[,<arg>...]" + CRC32(texto, 4 bytes LE) + '\n'
```

- Filtro de aceptación: `CRC ok · dst == cfg.nodeAddr || dst == 255 · cfg.masterAddr == 0 || src == cfg.masterAddr`.
- Comandos: `RD → ST,<a1..a4>,<d1..d4>,<o1..o4> · WR,<r1..r4> (0/1/-) · WP,<idx1..4>,<ms> · PING → PONG,<uptime_s>,<rssi>`.
- Errores: `ERR,<FMT|CMD|DIS|RANGE>`.
- Detalle completo en `PROTOCOL.md`.

---

## 5. Cómo extender

**Añadir un comando LoRa:** en `lora_proto.cpp::handleFrame()`, añade una rama `else if (!strcmp(t_cmd, "XX"))`, parsea args con `strtok_r(nullptr, ",", &save)` y responde con `reply(src, t_seq, body)`. Documenta en `PROTOCOL.md`.

**Añadir un campo de configuración:**

1. Campo en `struct NodeConfig (node_config.h)`.
2. Default en `configFactory()` y **sube** `CFG_MAGIC` en `node_config.cpp` (invalida blobs viejos con layout distinto).
3. Input en el formulario (`portal.cpp::buildPage()`) y parseo en `handleSave()`.
4. Si `nodeIO_master` comparte el módulo, copia el `.h/.cpp` idéntico allí.

**Cambiar la pantalla:** `drawStatusScreen()` / `drawPortalScreen()` en `main.cpp` (API `SSD1306Wire`: `clear()`, `setFont(ArialMT_Plain_10)`, `drawString()`, `drawXbm()`, `display()`).

---

## 6. Sincronización con nodeIO\_master

`io.{h,cpp}`, `node_config.{h,cpp}` y `portal.{h,cpp}` son **copias byte a byte** en ambos proyectos y hay que mantenerlas sincronizadas a mano (candidato a extraer a una `lib/` compartida). `NodeConfig` incluye campos solo-maestro (`targetAddr`, `pollMs`, `ackTimeoutMs`) que este firmware ignora. Específico del nodo: `lora_proto.*`. Específico del maestro: `lora_master.*`.

---

## 7. Notas de hardware

- **GPIO45 / GPIO46** (relés 3/4) son *strapping pins* del ESP32-S3 (VDD\_SPI / boot). `ioInit()` los deja en el estado seguro; el hardware externo no debe forzarlos durante el arranque/flasheo.
- A11–A14 = GPIO2–5 = **ADC1** (funciona con WiFi activo; ADC2 no se usa).
- `PIN_PWR_MGM` (GPIO36 = VEXT) se pone LOW en `ioInit()` para habilitar el riel externo (equivalente a `heltec_ve(true)`).

- `heltec_setup()` hace su propio `Serial.begin(115200)` e inicializa la OLED (`display.init/flipScreenVertically/setContrast`).